PHASE

13

File Systems: VFS, initrd and KaganFS

Turn a pile of numbered blocks into named files and directories, behind one interface that also covers devices and a read-only boot image

Depends on	Phase 12: block devices and the buffer cache; Phase 11: processes, `fork/exec`; Phase 9: mutexes
Delivers	A virtual file system layer (in-memory inodes, open-file objects, per-process descriptor tables, path lookup, mount points), a read-only root file system from a tar archive, `/dev` with `tty`, `null`, `zero` and disks, and KaganFS: a writable on-disk file system in the MINIX V3 format that Linux's `fsck.minix` accepts as clean. System calls: `open`, `close`, `read`, `write`, `lseek`, `dup`, `dup2`, `stat`, `fstat`, `getdents`, `mkdir`, `unlink`, `chdir`, `getcwd`.

1 Scope and motivation

A disk is a very long array of blocks. Users want named files in a tree of directories that they can open, read, grow and delete, and that survive crashes. The **file system** bridges the two, and it is the largest subsystem in Kagan. It uses nearly everything from before: the buffer cache and block devices (Phase 12), processes and `fork` semantics (Phase 11), user-pointer checks (Phase 10), and mutexes, because file-system operations sleep on disk I/O while holding locks (Phase 9).

The phase has two halves. The **VFS** (virtual file system) is the file-system-independent layer: file descriptors, open files, path names, mount points. It is what makes "everything is a file" possible in UNIX, since a terminal, a disk and a regular file all look the same to `read`. Below it sit the concrete file systems: first a trivial read-only one for booting, then **KaganFS**, a real on-disk format with inodes, bitmaps and indirect blocks. KaganFS uses the MINIX V3 layout from OSDI, so the book's figures describe a KaganFS disk exactly, and Linux can check KaganFS images and even mount them.

2 Background

2.1 Files, directories and descriptors

OSTEP chapter 39 describes the interface. A **file** is an array of bytes with a low-level name, its **inode number**. A **directory** is a file whose contents are a list of (name, inode number) pairs, and directories nest into a tree. A process doesn't use inodes directly; `open` returns a small integer, the **file descriptor**, which indexes the process's descriptor table. Three levels of objects are involved (Figure 1), and keeping them apart is the key to getting `fork` and `dup` right. The descriptor table is per process. An **open file** records the current offset and access mode and can be shared, so after `fork` the parent and child share one offset. An **in-memory inode** exists once per file no matter how many times it is open.

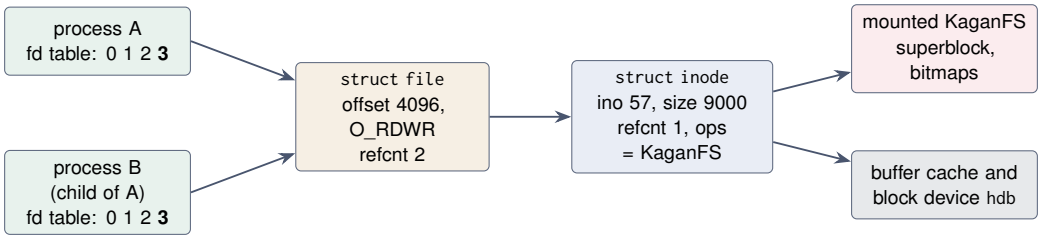


Figure 1: After `fork`, descriptor 3 in both processes refers to the same open file, and so to the same offset.

2.2 How an on-disk file system is laid out

OSTEP chapter 40 builds a very simple file system, *vsfs*, and KaganFS is its real-world sibling. The disk is divided into regions: a **superblock** describing the whole file system, an **inode bitmap** and a **zone bitmap** marking which inodes and data blocks are in use, the **inode table**, and the **data** region (Figure 2). MINIX calls data blocks *zones*; with the settings KaganFS uses, a zone is exactly one 1 KiB block (OSDI §5.6.2).

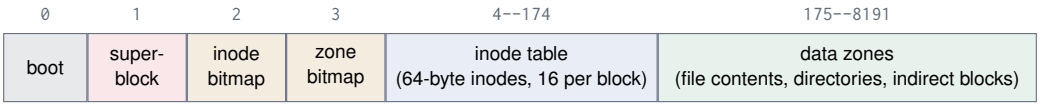


Figure 2: KaganFS on an 8 MiB image, exactly as `mkfs.minix -3` lays it out: 2736 inodes, first data zone 175. Numbers are 1 KiB block numbers.

Structure	Format (all little-endian)
superblock (at byte 1024)	`u32 s_ninodes; u16 pad; u16 s_imap_blocks; u16 s_zmap_blocks; u16 s_firstdatazone; u16 s_log_zone_size; u16 pad; u32 s_max_size; u32 s_zones; u16 s_magic (0x4D5A); u16 pad; u16 s_blocksize (1024); u8 s_disk_version`
inode (64 bytes)	`u16 i_mode; u16 i_nlinks; u16 i_uid; u16 i_gid; u32 i_size; u32 i_atime, i_mtime, i_ctime; u32 i_zone[10]`
directory entry (64 bytes)	`u32 inode; char name[60]` (name padded with zeros; inode 0 means an empty slot)
bitmaps	bit n of the inode map is inode n; bit n of the zone map is zone $n + $ `s_firstdatazone` $- 1$; bit 0 of both is reserved and always set

2.3 Finding a file's data

An inode has ten zone pointers (Figure 3). The first seven point directly at data zones, so small files need nothing else. Pointer 7 points to a **single indirect** block holding 256 more zone numbers, and pointer 8 to a **double indirect** block holding 256 pointers to blocks of 256 zone numbers each. The tenth pointer (triple indirect) is reserved and KaganFS doesn't use it. OSTEP §40.3 and OSDI §5.6.4 explain the scheme, and MOS §4.3.2 compares it with the alternatives: contiguous allocation, linked lists and FAT.

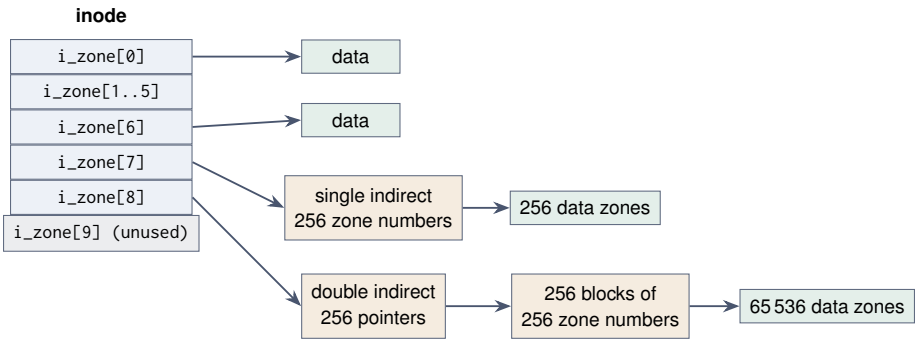


Figure 3: Zone pointers in a KaganFS inode, with 1 KiB zones and 4-byte zone numbers.

2.4 Path lookup, and what it costs

To open `/bin/hello`, the kernel starts at the root inode (inode 1), reads the root directory's data and searches it for `bin`, reads inode `bin`, reads its data and searches for `hello`, then reads inode `hello`. OSTEP §40.6 draws this as a timeline of disk reads, and it is the best way to understand why caching matters: without the buffer cache, every `open` costs several disk reads just to walk the path.

2.5 Crashes

Creating a file changes several blocks: the inode bitmap, the inode, the directory, maybe the zone bitmap and a data block. If the machine stops half-way, the file system is left inconsistent: an inode marked used that no directory points to, or a zone used by two files. OSTEP chapter 42 presents the two classic answers. **fsck** scans the whole disk after a crash and repairs it, which is simple but slow. **Journaling** writes a description of the change to a log first and replays it after a crash. KaganFS relies on `fsck.minix` on the host for now and orders its writes carefully (allocate, then initialise, then link), so a crash leaves at worst leaked blocks instead of corruption. A journal is listed under Extensions.

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	LOB	Ch. 12 *File Systems*: a read-only file system, inodes, a virtual file system	69–70	The route of this phase in two pages.
PRIMARY	OSTEP	Ch. 39 *Interlude: Files and Directories*	457–476	The interface: descriptors, offsets, `fork` and `dup` sharing, links.
PRIMARY	OSTEP	Ch. 40 *File System Implementation* (vsfs, inodes, multi-level index, the access-path timeline, caching)	477–494	The design of KaganFS in miniature.
PRIMARY	OSTEP	Ch. 42 *Crash Consistency: FSCK and Journaling*	507–526	Why write order matters.
PRIMARY	OSDI	§5.6 *Overview of the MINIX 3 File System*: messages, layout, bitmaps, i-nodes, block cache, directories and paths, descriptors, locking, pipes, the READ walkthrough	548–566	The KaganFS on-disk format, explained by its designers.
SECONDARY	OSDI	§5.7 *Implementation of the MINIX 3 File System*	566–606	Working code for every operation in Tasks 13.5–13.8.
SECONDARY	MOS	§4.1–4.3 *Files*, *Directories*, *File-System Implementation* (incl. journaling and VFS)	265–299	Allocation methods compared; VFS in §4.3.7.
SECONDARY	MOS	§4.5 *Example File Systems* (MS-DOS FAT, UNIX V7)	320–331	Two real designs to compare with KaganFS.
SECONDARY	OSC	Ch. 13 *File-System Interface*; §14.1–14.5; §15.5 *Virtual File Systems*	529–582, 603–605	A thorough second treatment.
OPTIONAL	OSTEP	Ch. 41 *Locality and the Fast File System*; Ch. 43 *Log-structured File Systems*	495–506, 527–542	How layout decisions affect speed.
OPTIONAL	MOS	§4.4 *File-System Management and Optimization*	299–320	Block size, free-space tracking, consistency checks, caching.
SECONDARY	LKD	Ch. 13, *The Virtual Filesystem*	261–288	Superblock, inode, dentry and file objects: the model for the Kagan VFS.

Prior art: MINIX 3

The MINIX file system server in `servers/fs/` implements the exact format KaganFS uses and serves as a reference implementation: the superblock and bitmap allocation (`alloc_bit`, `free_bit`) in `super.c (23300)`; the in-memory inode table (`get_inode`, `put_inode`, `alloc_inode`) in `inode.c (22900)`; path lookup (`eat_path`, `advance`, `search_dir`) in `path.c (26300)`; file position to zone mapping (`read_map`) in `read.c (25000)`; `open` and `mkdir` in `open.c (24500)`; descriptors in `filedes.c (23700)`. OSDI §5.6.10 traces one `read` call through all of it.

Linux, for comparison. LKD Linux's VFS has four main objects: the superblock, the inode, the **dentry** (a cached path component) and the file. The dentry cache is what makes repeated path lookups fast; the Kagan VFS has the other three.

4 Implementation

Task 13.1 The VFS objects

Define the three core structures. `struct inode` is the in-memory inode, file-system independent, with a table of operations that each file system fills in. `struct file` is an open file. A mounted file system is a `struct superblock` with its root inode. Keep in-memory inodes in a table keyed by (superblock, inode number), so each file has exactly one, with a reference count.

include/vfs.h

```
struct inode_ops {
    int     (*lookup)(struct inode *dir, const char *name, struct inode **out);
    int     (*create)(struct inode *dir, const char *name, uint16_t mode, struct inode **out);
    int     (*mkdir)(struct inode *dir, const char *name, uint16_t mode);
    int     (*unlink)(struct inode *dir, const char *name);
    int     (*readdir)(struct inode *dir, uint32_t index, struct dirent *out);
    int32_t (*read)(struct inode *ino, void *buf, uint32_t n, uint32_t off);
    int32_t (*write)(struct inode *ino, const void *buf, uint32_t n, uint32_t off);
    int     (*truncate)(struct inode *ino, uint32_t size);
};

struct inode {
    uint32_t                ino;        /* number inside its file system        */
    uint16_t                mode;       /* type bits (dir, file, device) + rwx  */
    uint32_t                size;
    int                     refcnt;     /* in-memory references                 */
    struct superblock      *sb;
    const struct inode_ops *ops;
    struct mutex            lock;       /* held during operations that may sleep */
    struct inode           *mounted;    /* root of a file system mounted here   */
    void                   *priv;       /* file-system specific data            */
};

struct file {
    struct inode *inode;
    uint32_t      offset;
    int           flags;                /* O_RDONLY, O_WRONLY, O_RDWR, O_APPEND */
    int           refcnt;               /* descriptors pointing here            */
};
```

Give the VFS one place where access is decided: `int vfs_permission(struct inode *ino, int mask)`, with `mask` made of `MAY_READ`, `MAY_WRITE` and `MAY_EXEC`. Call it from every VFS operation before the file system does any work: `open` (with the mask its flags ask for), `execve`, `chdir`, path lookup (`MAY_EXEC` on each directory it passes through), and `create`, `mkdir` and `unlink` (`MAY_WRITE` on the parent). There are no users yet, so it returns 0; Phase 15 fills in the body, and no call site has to change.

Task 13.2 Descriptors and the basic calls

Give each process a table of 64 `struct file` pointers (`OPEN_MAX`). `open` resolves the path, allocates a `struct file` and returns the lowest free descriptor. `read` and `write` copy through a kernel buffer with `copy_to_user`/`copy_from_user` and advance the offset. `close` drops a reference. `dup` and `dup2` copy a pointer and increment the count. `fork` copies the whole table, incrementing every count; `exec` keeps it, except the descriptors marked close-on-exec; `exit` closes everything. The close-on-exec flag belongs to the descriptor, not to the `struct file`, so keep it in a 64-bit mask next to the table: `open` sets the bit when its flags contain `O_CLOEXEC`, `fcntl(fd, F_SETFD, FD_CLOEXEC)` sets or clears it, `dup` and `dup2` clear it for the new descriptor, `fork` copies the mask, and `exec` closes every marked descriptor once the new program has loaded. A program that runs another one under a different identity, such as `login` in Phase 15, uses it so that its open files do not leak into the child. Keep using the Linux i386 numbers.

Call	No.	Call	No.	Call	No.
`open`	5	`lseek`	19	`stat`	106
`close`	6	`mkdir`	39	`fstat`	108
`unlink`	10	`dup`	41	`getdents`	141
`chdir`	12	`dup2`	63	`getcwd`	183
`fcntl`	55				

Task 13.3 Devices as files

Write a tiny `devfs` and mount it at `/dev`. Its inodes carry device operations instead of disk data: `/dev/tty` reads a line from the keyboard and writes to the console; `/dev/null` discards writes and returns end-of-file; `/dev/zero` returns zeros; `/dev/hda` and `/dev/hdb` read and write their block devices. When the kernel starts `init`, it opens `/dev/tty` three times, so `init` and everything it starts inherit descriptors 0, 1 and 2. `sys_write` to descriptor 1 now goes through the VFS like any other file.

Task 13.4 A read-only root from a tar archive

The simplest file system to parse is a tar archive in the POSIX *ustar* format, so use one as the initial root. Each file is a 512-byte header followed by its contents padded to a multiple of 512 bytes, and the archive ends with two zero blocks. The header holds the name (100 bytes at offset 0, plus a prefix of 155 bytes at offset 345), the size as an octal ASCII string (12 bytes at offset 124), the type (one byte at 156: `'0'` for a file, `'5'` for a directory) and the magic `"ustar"` at offset 257. Build the archive from a `rootfs/` directory in the Makefile, load it as a boot module, and mount it at `/`. Now `execve` can take real paths.

Treat the header as untrusted input, and keep the parsing in `lib/tar.c`, separate from the VFS glue, so it can also be compiled for the host (Task 13.10). Verify the checksum: the octal number in the 8 bytes at offset 148 must equal the sum of all 512 header bytes as unsigned values, with the checksum field itself counted as eight spaces. Parse every octal field with a range check, and reject a file whose contents would end past the end of the archive.

lib/tar.c (octal fields)

```
/* Parse an octal field of n bytes (leading spaces, then digits, then NUL or space).
 * Returns 0, or -EINVAL for a bad character or a value that does not fit in 32 bits. */
int tar_octal(const char *f, size_t n, uint32_t *out)
{
    size_t i = 0;
    uint32_t v = 0;
    while (i < n && f[i] == ' ')
        i++;
    for (; i < n && f[i] >= '0' && f[i] <= '7'; i++) {
```

```
        if (v > (UINT32_MAX >> 3))
            return -EINVAL;                     /* one more digit would overflow */
        v = (v << 3) | (uint32_t)(f[i] - '0');
    }
    if (i < n && f[i] != '\0' && f[i] != ' ')
        return -EINVAL;
    *out = v;
    return 0;
}
```

Makefile (root archive)

```
$(BUILD)/initrd.tar: $(USER_PROGS) $(shell find rootfs -type f)
    mkdir -p rootfs/bin && cp $(USER_PROGS) rootfs/bin/
    tar --format=ustar -cf $@ -C rootfs .
```

Verify: `execve("/bin/hello", ...)` loads the program from the archive, and `cat /etc/motd` (from the monitor for now) prints a file you put in `rootfs/etc/`. An archive with one header byte changed is refused because of its checksum, and a size of `77777777777` is rejected instead of wrapping around.

Task 13.5 Path lookup

Write `namei(path, &inode)`. Start at the root for absolute paths and at the process's current directory otherwise, then take one component at a time: skip empty components (`//`), stay put on `.`, and call the directory's `lookup` for everything else, including `...` When the result has a file system mounted on it, continue at that file system's root; when `..` is taken at the root of a mounted file system, go to the mount point's parent. Also write `nameiparent` (the directory and the last component), which `create`, `mkdir` and `unlink` need. Add `chdir` and `getcwd`.

Verify: `/bin/../etc/./motd`, `//etc//motd` and a relative `etc/motd` from `/` all resolve to the same inode; `/dev/..` is `/`.

Task 13.6 KaganFS, reading

Create a file-system image on the host and attach it to QEMU as the primary slave (`hdb`); your ATA driver needs to select the slave with `0xF0` in the drive register. Mount it at `/mnt`. Read the superblock through the buffer cache and check the magic and block size; read inodes by number (inode n is in block $2 + \text{imap} + \text{zmap} + (n-1)/16$, at offset $((n-1) \bmod 16) \times 64$); map a file offset to a zone through the direct, single and double indirect pointers; search directories entry by entry.

The image may come from anywhere, so check everything on it before you use it as an index. At mount, refuse with `-EINVAL` unless the superblock is consistent: `s_ninodes` and `s_zones` are not zero; the inode bitmap has room for `s_ninodes`$+1$ bits and the zone bitmap for `s_zones`$-$`s_firstdatazone`$+1$ bits (8192 bits per block); `s_firstdatazone` lies after the inode table and before `s_zones`; `s_zones` fits on the device; and `s_log_zone_size` is 0. After mount, check on every use: an inode number taken from a directory entry must satisfy $1 \le n \le$ `s_ninodes`, and a zone number taken from an inode or an indirect block must satisfy `s_firstdatazone` $\le z <$ `s_zones` (0 means a hole). Anything else returns `-EIO` and logs the inode and the bad value; nothing on the disk may make the kernel read or write outside a buffer. Keep these checks and the decoding of on-disk structures in `lib/kaganfs_parse.c`, without kernel calls, so the host can compile it too.

terminal (host)

```
dd if=/dev/zero of=build/kaganfs.img bs=1024 count=32768     # 32 MiB
mkfs.minix -3 build/kaganfs.img                               # MINIX V3 layout, 1 KiB blocks
sudo mount -o loop build/kaganfs.img /mnt                     # optional: put files on it
sudo cp -r rootfs/* /mnt && sudo umount /mnt
qemu-system-i386 ... -drive file=build/kaganfs.img,format=raw,if=ide,index=1,media=disk
```

Verify: `ls /mnt` (with the monitor, or a test program using `getdents`) lists what you copied from Linux, and reading a file larger than 263 KiB gives the right contents, which exercises the double indirect block. Mounting an image whose `s_ninodes` is 0 fails with `-EINVAL`, and reading a file whose inode points to a zone past `s_zones` fails with `-EIO`; neither panics.

Task 13.7 KaganFS, writing

Now allocation. Find a free inode by scanning the inode bitmap; find a free zone by scanning the zone bitmap and remember the offset: bit n is zone $n +$ `s_firstdatazone` $- 1$. Implement `create` (allocate and initialise an inode, then add a directory entry), `write` (allocate zones and indirect blocks as the file grows, zeroing new blocks), `mkdir` (a new directory containing `.` and `..`, with the parent's link count incremented), `unlink` (remove the entry, decrement the link count, and free the inode and its zones when the count reaches zero and nobody has the file open), and `truncate`. Do every change through the buffer cache, marking buffers dirty, and order the writes so a crash leaves at worst leaked space: allocate, initialise, and only then link. Add a `sync` system call and flush all file systems at shutdown.

Verify: After Kagan creates, writes and deletes files, shut it down cleanly and run `fsck.minix -f build/kaganfs.img` on the host: it reports no errors and exits with status 0.

Task 13.8 stat, fstat and getdents

Define `struct stat` (device, inode, mode, link count, size, times) and `struct dirent` (inode number, name), and implement the three calls through the inode operations. `memset` each structure to zero before you fill it: the compiler leaves padding between fields and after a short name, and without the `memset` those bytes carry old kernel stack contents to the user. These are what `ls` will use in Phase 14.

Task 13.9 Tests

Add these to `make test`, running on a fresh image each time; after the run, `make test` also runs `fsck.minix` on the image.

Test	What must hold
`many_files`	create 200 files with random contents, read them back, delete them; free inode and zone counts return to their starting values
`big_file`	write and verify a 2 MiB file (well into the double indirect range), then truncate it to 0
`dirs`	nested `mkdir`, removing directories, link counts of parents
`paths`	`.`, `..`, `//`, relative paths, crossing into and out of `/mnt` and `/dev`
`fd_sharing`	after `fork`, parent and child share an offset; after `dup2`, two descriptors share one; closing one leaves the other working
`readonly_root`	writing to the tar root fails with `-EROFS`
`bad_images`	every image in `tests/data/` (bad superblock, inode or zone numbers out of range, bad tar checksum) gives `-EINVAL` or `-EIO`, never a panic
`cloexec`	a descriptor opened with `O_CLOEXEC` is closed in the program started by `exec`; one without it stays open

Also test an unclean shutdown. A host script starts QEMU with a program that creates, writes and deletes files in a loop, kills QEMU with `kill -9` after a random number of seconds, runs `fsck.minix -a` on the image, and boots again. `fsck.minix` must be able to repair the image, and the next mount must succeed without a panic; run it twenty times. Finally, extend `user/bin/fuzz` from Phase 11 with the file calls: random and over-long paths, paths through regular files and with many `..`, offsets for `lseek` near 2^{31}, descriptors that are closed, out of range or negative, and random flags for `open`.

Task 13.10 Fuzz the parsers on the host

`lib/kaganfs_parse.c` and `lib/tar.c` use nothing from the kernel, so compile them for the host and feed them random input much faster than QEMU could. Write one fuzz target per parser in `tests/host/`: the tar target walks an archive given as a byte buffer; the KaganFS target treats the buffer as an image, validates the superblock, and then reads every inode and follows every zone pointer it finds. Build them with libFuzzer (`clang -fsanitize=fuzzer,address,undefined`) or with AFL++. Seed the corpus with small images made by `mkfs.minix` and a few files, and with the `initrd.tar` of your build.

tests/host/fuzz_tar.c

```
#include <stddef.h>
#include <stdint.h>
#include "tar.h"

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
{
    struct tar_entry e;
    size_t off = 0;
    while (tar_next(data, size, &off, &e) == 0)    /* every field checked inside */
        ;
    return 0;
}
```

Every crash or sanitizer report the fuzzer finds is a bug in the parser. Save the input that caused it in `tests/data/`, fix the bug, and add the file to the `bad_images` ktest so it can never come back.

Verify: Each fuzz target runs for one hour without a crash or a sanitizer report, and every file in `tests/data/` passes `bad_images`.

5 Testing and debugging

When the on-disk format goes wrong, compare with what Linux sees. Mount the image read-only on the host (`sudo mount -o loop,ro`) and look around, run `fsck.minix -fsv` for a detailed report, and dump raw blocks with `hexdump -C -s $((BLOCK*1024)) -n 1024 build/kaganfs.img`.

Symptom	Likely cause	Resolution
`fsck.minix` reports zones marked in use but not used, or the reverse	The zone bitmap offset is wrong	Bit n is zone $n + \texttt{s_firstdatazone} - 1$, not zone n.
Directory listing shows garbage names or skips entries	Inode 0 slots not skipped, or names treated as NUL-terminated when they are exactly 60 bytes	Skip entries with inode 0; copy at most 60 bytes and terminate yourself.
Wrong inode read	Inode numbers start at 1	Use $n - 1$ when computing the block and offset.
Writes vanish after reboot	Buffers not marked dirty, or no `sync` at shutdown	`bdirty` after every change; flush in the shutdown path.
After `fork`, the two processes read the file independently	The descriptor table was deep-copied into new `struct file` objects	Copy pointers and increment counts; they must share the offset.
Deadlock while creating a file	A directory inode lock and a buffer lock taken in different orders in different paths	Add the file-system locks to the lock order from Phase 9: inode, then buffer.

6 Exit criteria

Exit criteria: Files and directories

- ☐ The VFS resolves paths across `/` (tar), `/dev` (devfs) and `/mnt` (KaganFS).
- ☐ Every process has descriptors 0–2 on `/dev/tty`; `fork`, `dup2` and `exec` handle descriptors correctly.
- ☐ Programs are loaded by path from the file system.
- ☐ KaganFS reads and writes files and directories, and `fsck.minix -f` on the host finds no errors.
- ☐ All file-system tests pass, including the 2 MiB file.
- ☐ The superblock is validated at mount, every inode and zone number is checked on use, and tar headers have their checksum and sizes checked.
- ☐ The KaganFS and tar host fuzz targets run for an hour without a crash, and every crash they ever found is a ktest.
- ☐ Every VFS operation calls `vfs_permission`; `exec` closes `O_CLOEXEC` descriptors; `stat` and `dirent` are zeroed before they are filled.
- ☐ After a QEMU kill during writes, `fsck.minix` repairs the image and the next mount does not panic; `fuzz` covers the file calls.

Review questions

1. List the disk reads needed to open and read `/home/turk/notes.txt` with an empty cache, following OSTEP §40.6.
2. What is the largest file KaganFS can hold with seven direct zones, one single and one double indirect block, 1 KiB zones and 4-byte zone numbers?
3. What is the difference between a file descriptor, an open file and an inode? Which of them does `fork` copy?
4. What does the VFS layer provide? Name three things in Kagan that would be harder without it.
5. Why is crash consistency hard? In what order should `create` write its blocks, and why?
6. What is the difference between a hard link and a symbolic link? How does the link count relate to `unlink`?

7 Extensions

Make KaganFS the root file system and keep the tar archive only for early boot. Add a simple metadata journal (OSTEP §42.3): write the blocks of each operation to a log region first, then a commit block, then apply them, and replay the log at mount. Write a FAT16 driver (MOS §4.5.1) so Kagan can read USB-stick images made on any computer. Or add symbolic links, `rename`, and file permissions checked against the process's user ID, which Phase 15 builds on.

Next: **Phase 14 Userland: libc, Shell and Utilities** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and **OSTEP** are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.